

VISIONQUEUE

Team Work Allocation & Role Guide

Six-person development plan for the Computer Vision Laboratory project

Principle	How the team works
Architecture first	Roles follow software responsibilities, not equal-sized modules.
Parallel development	Each person gets a clear area with defined inputs/outputs.
No one is blocked	Frontend, backend, database and testing can use mock/simulated CV data.
Shared integration	The project is integrated continuously; it is not left until the last day.
Knowledge sharing	Each member understands what their work does, why it matters, and what it connects to.
Flexible support	Members help across boundaries when integration or testing requires it.

This document is separate from the TDD. The TDD describes the system architecture; this document explains how the six members can work on that architecture without making anyone feel secondary or dependent on one machine.

1. Project and Team Overview

VisionQueue is a local, single-camera intelligent queue and crowd management system. The core pipeline is camera input → computer vision → tracking/counting → analytics → backend → live dashboard. The team allocation mirrors these technical responsibilities while deliberately keeping interfaces simple.

The goal is not to create six isolated mini-projects. The goal is to create one integrated product in which each member owns a meaningful part and understands how that part contributes to the complete system.

1.1 End-to-end picture

Layer	Main responsibility	Primary contributors
Camera / CV input	Capture and prepare frames	Ram, Shakeeb
Detection	Person/face detection	Ram, Shakeeb
Tracking / counting	Track people and calculate counts	Ram, Shakeeb
Analytics / state	Occupancy, crowd state, alerts	Ram, Shakeeb, Vijay
Backend / API	Expose system data and configuration	Vijay
Database	Persist sessions/events/history	Manoj
Frontend	Dashboard and operator interaction	Phaneeth
Testing / integration	Verify contracts and whole system	Roony + everyone

2. Ram Samhith — Computer Vision & System Integration

Computer vision is the part that converts raw camera frames into meaningful information. The integration work connects the CV output to the rest of the product so that detection, tracking, counting, analytics, backend and dashboard behave as one system.

What this person will do

- Set up and maintain the main CV pipeline.
- Integrate OpenCV camera input.
- Integrate the person detector and face detector.
- Connect detection output to tracking.
- Integrate tracking with ROI and counting logic.
- Help define the CV-to-application data contract.
- Keep the CV engine modular so it can run independently of the frontend/database.
- Coordinate end-to-end integration of the major modules.
- Investigate CV/runtime problems that appear during integration.
- Support final performance and real-time validation.

What this work produces and connects to

Area	Use in the project
Detection	Produces person/face detections from frames.
Tracking	Maintains person track IDs across frames.
Counting	Turns tracks into current/entry/exit/session counts.
Integration	Connects CV output to analytics and application state.
Validation	Confirms the whole pipeline actually works together.

Why this role is important

Why this matters: Without a reliable CV-to-application connection, the project is only a collection of separate modules. This work turns the computer-vision pipeline into an actual usable system.

3. Shaik Shakeeb — Computer Vision & Tracking

Tracking is what makes the system more than frame-by-frame object detection. It allows VisionQueue to understand that a person detected in consecutive frames is generally the same tracked object during a session.

What this person will do

- Work on person-detection integration and testing.
- Integrate and tune ByteTrack.
- Test tracking under multiple people and crossing paths.
- Investigate ID switches and temporary detection loss.
- Support ROI membership and line-crossing behavior.
- Help validate duplicate-count prevention.
- Create/reuse repeatable video clips for CV testing.
- Measure detection/tracking performance under realistic conditions.
- Document tracking limitations and useful parameter values.
- Support CPU/GPU inference validation where required.

What this work produces and connects to

Area	Use in the project
Tracking	Maintains temporary session-scoped IDs.
Counting support	Provides the identity continuity needed to count crossings.
CV testing	Finds conditions where tracking breaks down.
Performance	Helps keep the vision loop responsive.
Validation evidence	Provides measured results rather than guesses.

Why this role is important

Why this matters: A detector can say 'there are people in this frame.' Tracking helps the system reason about movement and prevents obvious repeated counting across consecutive frames.

4. B. Vijay Kumar Goud — Backend & API

The backend is the communication layer between the system's internal state and the dashboard. It provides stable interfaces so the frontend does not need to know anything about YOLO, OpenCV or ByteTrack.

What this person will do

- Build the FastAPI application structure.
- Implement REST endpoints for health, state, configuration, sessions and history.
- Implement the WebSocket live-state channel.
- Define and validate API request/response schemas.
- Connect application state to the frontend-facing contracts.
- Handle connection/reconnection behavior.
- Implement API validation and structured error responses.
- Keep heavy CV processing outside HTTP request handlers.
- Coordinate with the database layer for history/configuration.
- Provide mock API/WebSocket behavior for frontend development.

What this work produces and connects to

Area	Use in the project
REST API	Provides predictable application operations.

WebSocket	Pushes live counts, occupancy, crowd state and alerts.
Contracts	Lets frontend and backend work independently.
Health/status	Makes system problems visible instead of hiding them.
Integration	Connects CV/analytics outputs to the UI.

Why this role is important

Why this matters: The backend turns internal program state into a stable product interface. Without it, the frontend would become tightly coupled to the CV implementation.

5. Y. Manoj Kumar Reddy — Database & Data Management

The database provides persistence for information that should remain available after individual frames have disappeared, such as sessions, measurements, events and configuration.

What this person will do

- Design the SQLite database schema.
- Create sessions, measurements, events and configuration tables.
- Define relationships, constraints and indexes.
- Implement database access/repository functions.
- Use SQLite WAL appropriately.
- Keep database writes out of the critical per-frame CV loop.
- Implement asynchronous/queued persistence integration.
- Test restart and persistence behavior.
- Test history queries and data consistency.
- Coordinate database contracts with the backend.

What this work produces and connects to

Area	Use in the project
Sessions	Records monitoring runs.
Measurements	Stores aggregated historical values when Core+ is enabled.
Events	Stores alerts and important state transitions.
Configuration	Stores persistent camera/ROI/capacity settings.
Data quality	Ensures history is consistent and useful.

Why this role is important

Why this matters: The database gives the project memory. It allows the system to show what happened during a session instead of only displaying the current frame.

6. V. Phaneeth Kumar — Frontend & Dashboard

The frontend is the part the operator sees. It converts the system's live state into a clear interface that communicates current count, occupancy, crowd state, alerts and reliability.

What this person will do

- Build the React + TypeScript application structure.
- Create the main monitoring dashboard.
- Implement live count, occupancy and crowd-level displays.
- Implement system/camera/degraded/offline status indicators.

- Implement alert presentation.
- Integrate WebSocket live state.
- Use REST/TanStack Query for history and configuration data where appropriate.
- Build Core+ charts when enabled.
- Implement loading, error, offline and reconnecting states.
- Use mock API/WebSocket data so frontend work can proceed without the CV engine.

What this work produces and connects to

Area	Use in the project
Dashboard	Makes system information understandable to an operator.
Live state	Shows current system behavior.
Reliability UI	Prevents users from mistaking OFFLINE/UNSTABLE data for valid live data.
History	Makes Core+ measurements useful.
UX	Keeps the product practical rather than looking like a decorative AI demo.

Why this role is important

Why this matters: Even a technically correct backend/CV system is difficult to use if the operator cannot immediately understand its current state. The dashboard is the product's visible interface.

7. P. Rooney Williams — Testing & Integration

Testing makes sure individual modules work and, more importantly, that they continue to work when connected. This role focuses on repeatable validation rather than being a final-day 'tester.'

What this person will do

- Create the project test plan and test matrix.
- Write unit tests for important logic.
- Test API endpoints and validation.
- Test WebSocket connection/reconnection behavior.
- Test database operations.
- Use mock CV/state data for repeatable tests.
- Run end-to-end tests across CV → analytics → backend → frontend.
- Test camera disconnect/recovery.
- Test GPU fallback and degraded states.
- Test stale-frame and resource-pressure behavior.
- Maintain a list of known failures and regression tests.

What this work produces and connects to

Area	Use in the project
Unit testing	Catches logic errors early.
Integration testing	Confirms module contracts actually work together.
Failure testing	Checks that the system fails safely.
Regression testing	Prevents fixes in one module from breaking another.
Final validation	Provides evidence for the project's claims.

Why this role is important

Why this matters: VisionQueue has several interacting real-time components. Testing prevents the project from appearing to work in a simple demo while failing under realistic conditions.

8. Communication and Integration — Shared Responsibility

Integration is not assigned to one person. Every member is responsible for making their module communicate through the agreed contract. The main integration coordination is handled through the CV/system integration work, but the entire team participates.

8.1 Contract chain

Camera → CV → Tracking → Counting → Analytics → Application State → API/WebSocket → Frontend → Database

8.2 What each person must provide

Area	Must provide to the next layer
CV	Structured detections/tracks and health state.
Tracking/Counting	Track IDs, count events, current/unique count information.
Analytics	Occupancy, crowd level, alerts and system state.
Backend	Stable REST/WebSocket contracts.
Database	Persistent sessions/events/history/configuration.
Frontend	Clear operator presentation and user configuration.
Testing	Evidence that contracts behave correctly.

8.3 Integration rule

No module should depend on another member's internal implementation. It should depend on the agreed input/output contract. If the real CV engine is unavailable, the mock implementation should provide the same application-level data shape.

9. How the Team Can Work in Parallel

Phase	CV work	Backend/DB work	Frontend work	Testing/integration
Foundation	Camera + model/runtime setup	API skeleton + schema design	UI skeleton + mock state	Test structure + contracts
Core implementation	Detection + tracking + counting	REST + WebSocket + persistence	Live dashboard + states	Unit/API/WebSocket tests
Integration	Produce stable CV state	Connect CV/analytics state	Connect live API	End-to-end integration
Validation	CV accuracy/performance	API/DB performance	UI reliability	Failure/performance/soak tests
Finalization	Tune and document limitations	Harden interfaces	Polish usability	Regression + final evidence

9.1 Important advantage

The frontend, backend and database do not have to wait for the CV pipeline. They can use mock/simulated data. This prevents the entire team from becoming blocked by computer-vision setup or GPU availability.

10. What Everyone Should Understand

Nobody needs to master every technology. However, everyone should understand the system's basic flow and their own module's contract.

Everyone should know	Expected understanding
Overall pipeline	Camera → CV → tracking/counting → analytics → backend → dashboard.
Their module	Purpose, inputs, outputs and failure behavior.
Git/GitHub	Branch, commit, pull, merge/rebase as agreed by the team, and resolve conflicts.
API contracts	How structured data moves between modules.
Testing	How to run the project's tests and reproduce failures.
Integration	How their changes affect the adjacent modules.
Scope	Which features are V1 and which are future.

11. What Nobody Needs to Feel Pressured About

- Nobody is expected to know the entire stack immediately.
- Nobody needs to run the full GPU CV pipeline just to contribute.
- Frontend/backend/database members can work with mock data.
- CV members do not need to build the entire UI or database.
- Testing is not 'less important' work; it is what proves the other work functions together.
- Database work is not secondary; reliable history and configuration depend on it.
- Frontend work is not only visual design; it communicates system state and reliability to the operator.
- Integration is not an extra task that appears at the end; it should happen continuously.

12. Shared Rules for a Healthy Team

1. Agree on data contracts before implementing cross-module integrations.
2. Do not change an interface used by another module without discussing the impact.
3. Keep commits focused and descriptive.
4. Pull/rebase/merge frequently according to the team's Git workflow.
5. Open issues for blockers instead of silently waiting.
6. Use mock data when a dependency is not ready.
7. Do not commit model binaries, generated files or secrets unless explicitly agreed.
8. Record important architecture changes in the project documentation.
9. Test changes that affect another module.
10. Keep the P0 real-time system stable before adding Core+ features.

13. Integration Ownership Matrix

Integration point	Primary contributor	Supporting contributors
Camera → CV	Ram	Shakeeb
Detection → Tracking	Shakeeb	Ram
Tracking → Counting	Ram / Shakeeb	Roony
Counting → Analytics	Ram	Vijay
Analytics → Backend	Vijay	Ram
Backend → Database	Vijay	Manoj
Backend → Frontend	Vijay	Phaneeth
Frontend live-state handling	Phaneeth	Vijay
End-to-end testing	Roony	All
Final system integration	Ram	All

This matrix identifies the main collaboration points; it does not make one person solely responsible for the success of an integration. The adjacent contributors should review the contract together.

14. Suggested Deliverables by Member

Member	Concrete deliverables
Ram	CV pipeline, CV interfaces, counting integration, system integration notes, final CV/runtime validation.
Shakeeb	Tracking module, tracking test clips/results, counting/tracking validation, CV performance observations.
Vijay	FastAPI structure, REST endpoints, WebSocket service, schemas, backend mock interface.
Manoj	SQLite schema, repositories, persistence worker integration, database tests.
Phaneeth	React dashboard, reusable components, WebSocket client, history UI, UI state handling.
Roony	Test plan, unit/integration tests, failure tests, E2E checklist, regression results.

15. Definition of Team Completion

The team is not finished when each person says their module works independently. The project is finished when the modules work together and the resulting system satisfies the agreed V1 behavior.

11. Camera starts and produces usable frames.
12. Person and face detection run.
13. Tracking remains usable under representative conditions.
14. Current count and entry/exit behavior work.
15. Approximate session unique count behaves according to its documented limitation.
16. Occupancy and crowd state update correctly.
17. Alerts and reliability states behave correctly.
18. Backend REST/WebSocket contracts work.
19. Dashboard displays live state correctly.
20. Database persistence works without blocking the CV loop.
21. GPU fallback/degraded behavior is tested.
22. Camera disconnect/recovery is tested.
23. End-to-end and regression tests pass.

16. Final Team Principle

Every role is a real engineering responsibility. Computer vision creates the information; tracking and counting make it meaningful; analytics turns it into decisions; backend makes it accessible; database gives it memory; frontend makes it understandable; testing proves that the complete system is trustworthy. The project succeeds only when all of these pieces work together.